ASIC

Final System Verification Report
UART-Based Digital System

Supervised by: Eng. Ali Eltemsah

- # System Overview

The final system receives UART commands, processes them through the system controller, register file, ALU, and asynchronous FIFO, then transmits the result through UART TX.
Supported commands:

1. AA Write Register
2. BB Read Register
3. CC ALU Operation with New Operands
4. DD ALU Operation using Stored Operands

- # Testbench Setup

1. REF_CLK = 100 MHz
2. UART_CLK = 3.6864 MHz
3. UART Baud Rate = 115200
4. Even parity enabled
5. Testbench verifies RX, system control, register file, ALU, FIFO, and TX path

- # Waveform Signal Description

1. **test_name:** Shows the name of the currently running test case.
2. **status:** Indicates the current test status: running, PASS, or FAIL.
3. **expected:** The expected result of the current test.
4. **actual:** The actual result produced by the system.
5. **RX_IN:** UART serial input signal used to send commands and data to the system.
6. **data_valid:** Indicates that a complete valid byte has been received by the UART RX.
7. **p_data:** Parallel 8-bit data produced by the UART RX after deserialization.
8. **WrEn:** Register File write-enable signal.
9. **Address:** Register File address used for read or write operations.
10. **WrData:** Data written into the Register File.
11. **REG0–REG3:** Special Register File outputs used by the system for operands and UART configuration.
12. **TX_OUT:** UART serial output used to transmit read data or ALU results.
13. **prescale:** Current UART RX prescale configuration.

- # Test Cases1

Test Case 1 — Register Write Command (AA)

Command:

AA 04 0A

Operation:

Write 0x0A into register address 0x04.

Expected Result:

REG4 = 0x0A

Result: PASS

Waveform:

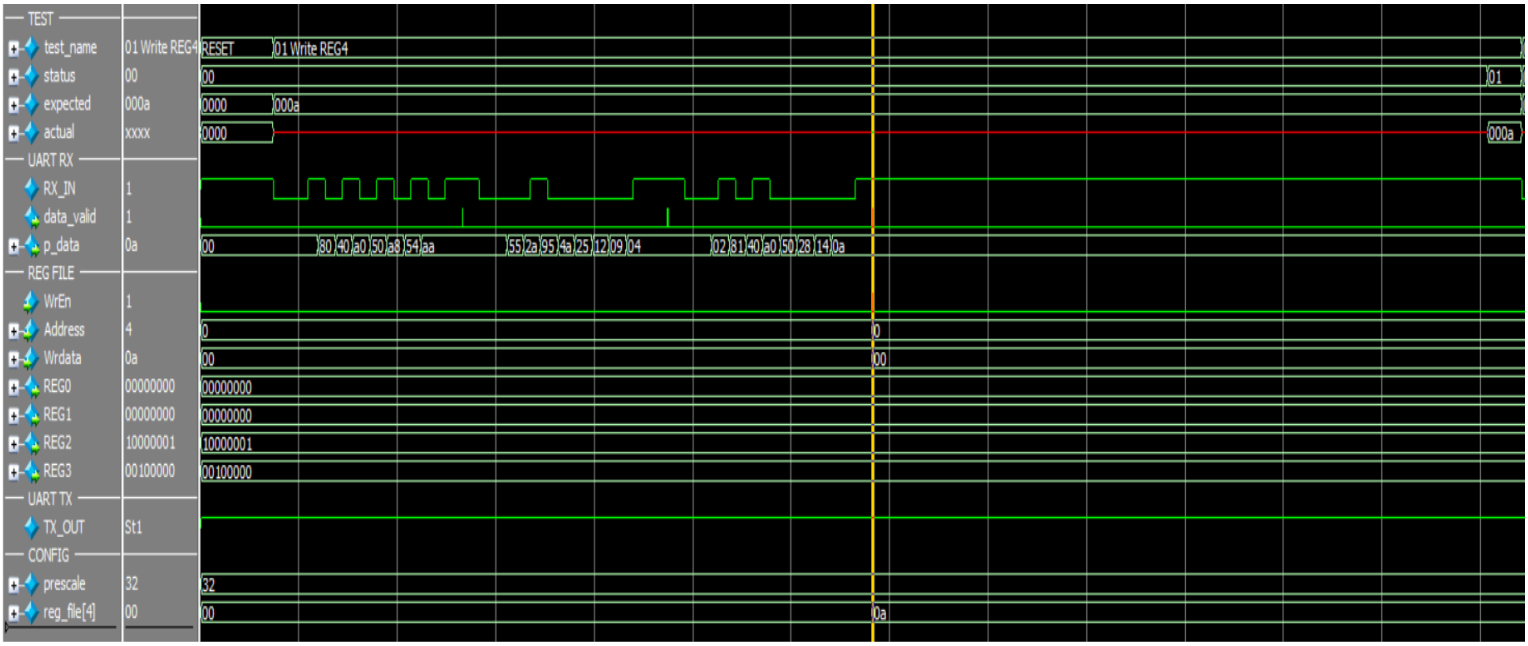


Figure 1 : Waveform showing the successful execution of the AA write command. Register address 0x04 is selected, 0x0A is written, and REG4 updates to 0x0A at the write clock edge.

- Test Cases2

Test Case 2 — Register Read Command (BB)

Command:

BB 04

Operation:

Read register address 0x04.

Expected Result:

0x0A

Result: PASS

Waveform:

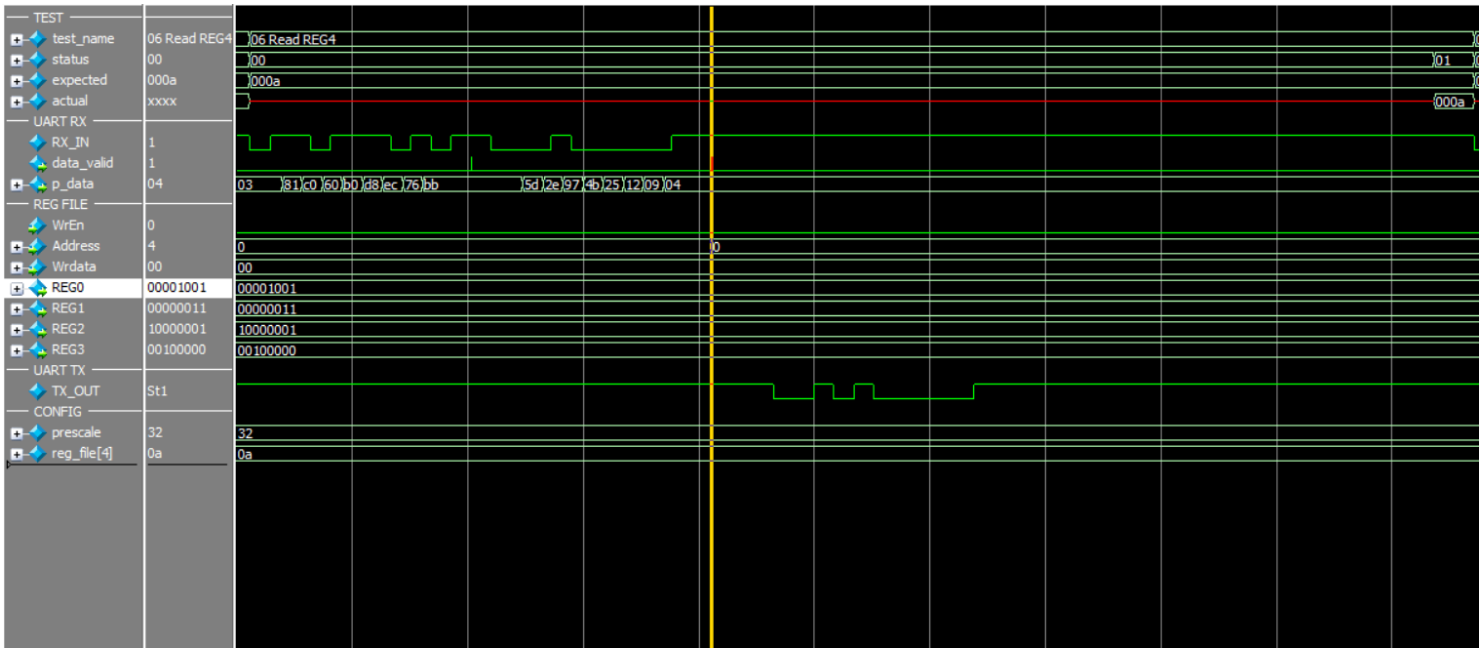


Figure 2 : Waveform showing the successful execution of the BB read command. Register address 0x04 is read and the stored value 0x0A is transmitted through UART TX.

- # Test Cases3

Test Case 3 — ALU Command with New Operands (CC)

Command:

CC 14 07 01

Operation:

20 - 7

Expected Result:

13 = 0x000D

UART Output:

0x0D 0x00

Result: PASS

Waveform:

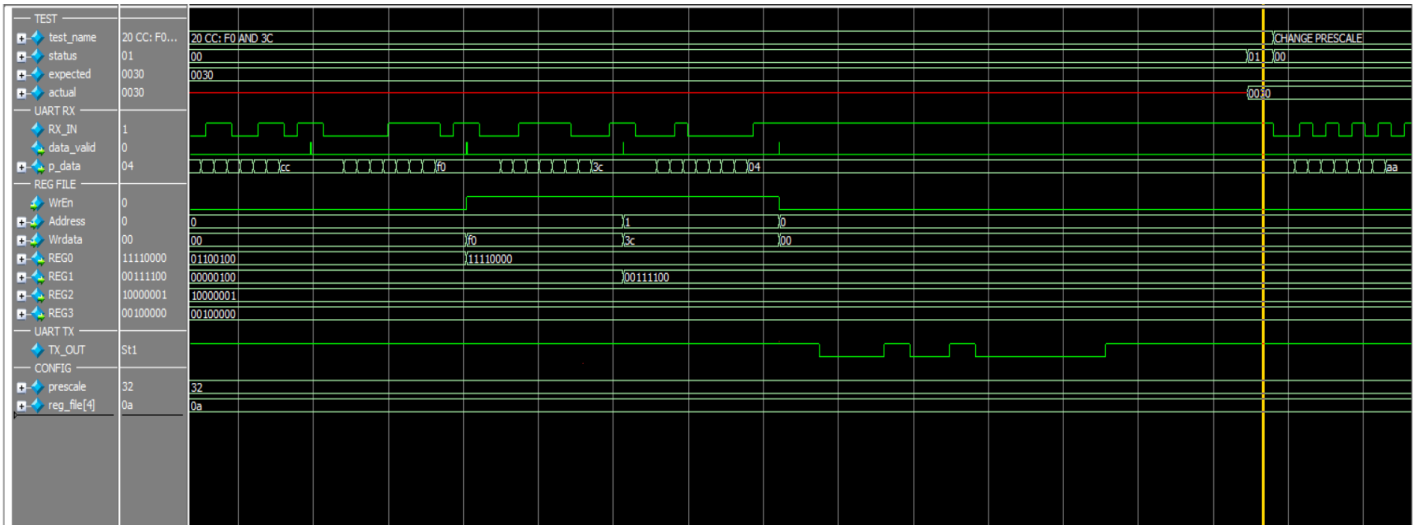


Figure 3 : Waveform showing the successful execution of the CC command. New operands are received through UART, the selected ALU operation is executed, and the result is transmitted through UART TX.

- # Test Cases4

Test Case 4 — ALU Command using Stored Operands (DD)

Command:

DD 02

Operation:

9 × 3

Expected Result:

27 = 0x001B

UART Output:

0x1B 0x00

Result: PASS

Waveform:

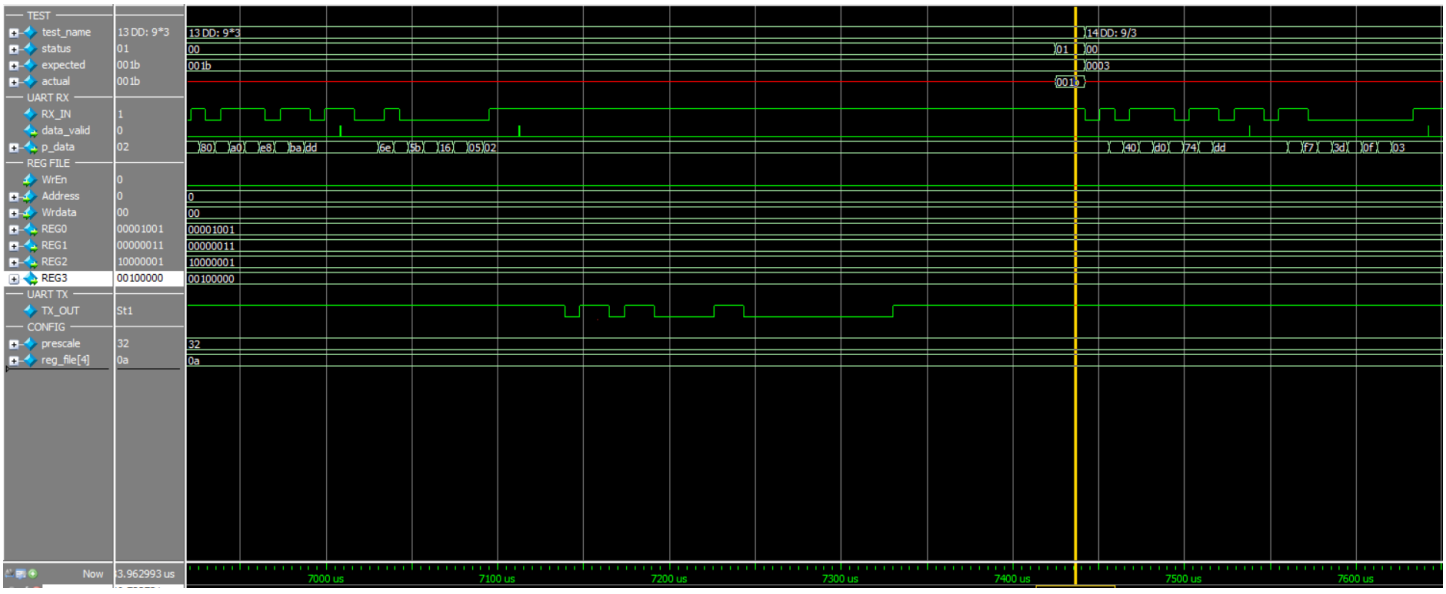


Figure 4 : Waveform showing the successful execution of the DD command. The ALU uses the operands stored in REG0 and REG1, performs the selected operation, and transmits the result through UART TX.

- LOG

```
#
# SIMPLE_28_V2 | 28 TESTS | PRESCALE 32 -> 16 -> 8
#
# TEST 1 | PRESCALE=32 | 01 Write REG4
# PASS | EXPECTED=000a ACTUAL=000a
#
# TEST 2 | PRESCALE=32 | 02 Write REG5
# PASS | EXPECTED=00ff ACTUAL=00ff
#
# TEST 3 | PRESCALE=32 | 03 Write REG15
# PASS | EXPECTED=0055 ACTUAL=0055
#
# TEST 4 | PRESCALE=32 | 04 Write operand A=9
# PASS | EXPECTED=0009 ACTUAL=0009
#
# TEST 5 | PRESCALE=32 | 05 Write operand B=3
# PASS | EXPECTED=0003 ACTUAL=0003
#
# TEST 6 | PRESCALE=32 | 06 Read REG4
#     TX byte = 0a
# PASS | EXPECTED=000a ACTUAL=000a
#
# TEST 7 | PRESCALE=32 | 07 Read REG5
#     TX byte = ff
# PASS | EXPECTED=00ff ACTUAL=00ff
#
# TEST 8 | PRESCALE=32 | 08 Read REG15
#     TX byte = 55
# PASS | EXPECTED=0055 ACTUAL=0055
#
# TEST 9 | PRESCALE=32 | 09 Read operand A
#     TX byte = 09
# PASS | EXPECTED=0009 ACTUAL=0009
#
# TEST 10 | PRESCALE=32 | 10 Read operand B
#     TX byte = 03
# PASS | EXPECTED=0003 ACTUAL=0003
#
# TEST 11 | PRESCALE=32 | 11 DD: 9+3
#     TX byte = 0c
#     TX byte = 00
# PASS | EXPECTED=000c ACTUAL=000c
#
# TEST 12 | PRESCALE=32 | 12 DD: 9-3
#     TX byte = 06
#     TX byte = 00
# PASS | EXPECTED=0006 ACTUAL=0006
#
# TEST 13 | PRESCALE=32 | 13 DD: 9*3
#     TX byte = 1b
#     TX byte = 00
# PASS | EXPECTED=001b ACTUAL=001b
#
# TEST 14 | PRESCALE=32 | 14 DD: 9/3
#     TX byte = 03
#     TX byte = 00
```

```
# PASS | EXPECTED=0003 ACTUAL=0003
#
# TEST 15 | PRESCALE=32 | 15 DD: 9 AND 3
#     TX byte = 01
#     TX byte = 00
# PASS | EXPECTED=0001 ACTUAL=0001
#
# TEST 16 | PRESCALE=32 | 16 CC: 200+100
#     TX byte = 2c
#     TX byte = 01
# PASS | EXPECTED=012c ACTUAL=012c
#
# TEST 17 | PRESCALE=32 | 17 CC: 20-7
#     TX byte = 0d
#     TX byte = 00
# PASS | EXPECTED=000d ACTUAL=000d
#
# TEST 18 | PRESCALE=32 | 18 CC: 20*20
#     TX byte = 90
#     TX byte = 01
# PASS | EXPECTED=0190 ACTUAL=0190
#
# TEST 19 | PRESCALE=32 | 19 CC: 100/4
#     TX byte = 19
#     TX byte = 00
# PASS | EXPECTED=0019 ACTUAL=0019
#
# TEST 20 | PRESCALE=32 | 20 CC: F0 AND 3C
#     TX byte = 30
#     TX byte = 00
# PASS | EXPECTED=0030 ACTUAL=0030
#
# ===== CHANGE PRESCALE: 32 -> 16 =====
# ===== ACTUAL PRESCALE=16 | REG2=41 =====
#
#
# TEST 21 | PRESCALE=16 | 21 PS16: Write REG4=A5
# PASS | EXPECTED=00a5 ACTUAL=00a5
#
# TEST 22 | PRESCALE=16 | 22 PS16: Read REG4
#     TX byte = a5
# PASS | EXPECTED=00a5 ACTUAL=00a5
#
# TEST 23 | PRESCALE=16 | 23 PS16: CC 20==20
#     TX byte = 01
#     TX byte = 00
# PASS | EXPECTED=0001 ACTUAL=0001
#
# TEST 24 | PRESCALE=16 | 24 PS16: DD 20>>1
#     TX byte = 0a
#     TX byte = 00
# PASS | EXPECTED=000a ACTUAL=000a
#
# ===== CHANGE PRESCALE: 16 -> 8 =====
# ===== ACTUAL PRESCALE=8 | REG2=21 =====
#
#
# TEST 25 | PRESCALE=8 | 25 PS8: Write REG15=5A
```

```
# PASS | EXPECTED=005a ACTUAL=005a
#
# TEST 26 | PRESCALE=8 | 26 PS8: Read REG15
#     TX byte = 5a
# PASS | EXPECTED=005a ACTUAL=005a
#
# TEST 27 | PRESCALE=8 | 27 PS8: CC 128<<1
#     TX byte = 00
#     TX byte = 01
# PASS | EXPECTED=0100 ACTUAL=0100
#
# TEST 28 | PRESCALE=8 | 28 PS8: DD 128>1
#     TX byte = 02
#     TX byte = 00
# PASS | EXPECTED=0002 ACTUAL=0002
#
# TOTAL=28 | PASS=28 | FAIL=0
# ALL 28 TESTS PASSED
```